

Name-Family Game Score Calculator

Technical Documentation

Author: AiPixelCode

Version: 1.0

June 2026

Project Overview

Name-Family Game Score Calculator is a Python application that automatically calculates players' scores in a Name-Family game. The program receives participants' answers, validates them against a predefined dataset of accepted words, and computes the final score for each player according to the game's scoring rules.

To improve maintainability and readability, the project separates game logic from its data source. Valid answers are stored in a CSV file, while the scoring rules are implemented within the application.

Project Structure

Name-Family-Game-Score-Calculator

```
├── game.py
├── game_data.csv
├── main.py
├── Documant.pdf
└── README.md
```

game.py: Contains the core game logic, including data loading, player registration, and score calculation.

game_data.csv: Stores the list of valid answers for each category.

main.py: Demonstrates how to use the project by registering sample participants and displaying the final scores.

README.md: Provides a quick guide for installing and running the project.

Program Workflow

The application executes in four main stages.

Step 1 – Load Valid Data

The application reads the `game\_data.csv` file and loads all valid answers into memory.

Step 2 – Register Participants

Each participant and their answers are added to the system. During this stage, answers are normalized and duplicate occurrences are counted.

Step 3 – Validate Answers

Each submitted answer is evaluated based on:

- Whether the answer is valid
- Whether the answer is unique or duplicated
- Whether at least one participant left that category unanswered

Step 4 – Calculate Scores

The appropriate score is assigned to every valid answer according to the scoring rules, and each participant's total score is calculated.

Data Structures

The project uses a small set of efficient data structures to manage the game state.

Structure	Purpose
`valid_words`	Stores valid answers for each category loaded from the CSV file.
`players`	Stores each participant and their submitted answers.
`answer_counts`	Tracks how many times each answer appears in every category.
`player_scores`	Stores the final score of every participant.
`categories_with_missing_answers`	Stores categories where at least one participant submitted an empty answer.

A **set** is used for ``categories_with_missing_answers`` to provide fast membership lookups during score calculation.

Main Functions

Function	Description
<code>`normalize(text)`</code>	Removes spaces from answers before comparison.
<code>`ready_up()`</code>	Loads the CSV file and initializes the application's data structures.
<code>`add_participant()`</code>	Registers a participant and records their answers.
<code>`calculate_all()`</code>	Calculates and returns the final scores for all participants.

Scoring Rules

Each answer is scored according to its validity, uniqueness, and whether the corresponding category contains any missing answers.

Answer Status	Score
Valid, unique, no missing answers	10
Valid, duplicated, no missing answers	5
Valid, unique, category contains missing answers	15
Valid, duplicated, category contains missing answers	10
Invalid or empty	0

Answer Normalization

Before comparing answers, all spaces are removed to avoid treating different writing styles as different answers.

For example:

Apple Pie

Apple Pie

Both values are considered identical during validation.

Example Usage

```
from game import ready_up, add_participant, calculate_all
ready_up()
add_participant(
    participant="Alice",
    answers={
        "name": "Alice",
        "family": "Anderson",
        "country": "Australia",
        "color": "Amber",
        "object": "Anchor",
        "food": "Apple Pie",
    },
)
scores = calculate_all()
print(scores)
```

Sample Output

```
=====
Final Scores
=====
Alice    65 pts
Bob      60 pts
Charlie  60 pts
David    40 pts
```

Project Strengths

- Modular design with clear separation between logic and data
- CSV-based dataset for managing valid answers
- Simple and readable implementation
- Easy to extend by modifying the dataset without changing the application logic
- Uses only Python's standard library
- Core logic can be reused in other applications

Current Limitations

- Valid answers must exist in the CSV dataset.
- Scoring rules are hardcoded and cannot be configured at runtime.
- The project provides no graphical user interface.
- Missing or malformed CSV files are not handled with custom error messages.

Possible Improvements

Future versions of the project could include:

- A graphical user interface (GUI)
- File- or form-based player input
- Exporting results to a file
- Configurable scoring rules
- Custom game categories
- A web-based version of the application

Conclusion

Name-Family Game Score Calculator is a compact yet well-structured Python project that automates the scoring process of a Name-Family game. By separating game logic from data and organizing the code into independent components, the project provides a clean, maintainable, and extensible implementation suitable for educational purposes and portfolio presentation.